



Object Oriented Programming in JavaScript

A Complete Beginner-Friendly Guide

Step-by-step • Plain language • Real examples • Practice exercises

How to use this guide

Read every section in order — each concept builds on the last.

Type every code example yourself in VS Code. Do not copy-paste.

Try the practice exercise after each section before moving on.

If something is confusing, re-read the analogy first, then the code.

01

What is Object Oriented Programming?

The Simple Explanation

So far you have been writing code as a list of instructions — one after another. That is called procedural programming. It works for small programs. But when your program grows big, it becomes very hard to manage.

Object Oriented Programming (OOP) is a different way of thinking. Instead of writing a list of instructions, you organise your code around objects.

What is an object? An object is a thing that has properties (data) and actions (behaviour) bundled together.

Real World Analogy

🚗 Think of a Car

A car has properties: colour, brand, speed, fuel level.

A car has actions (methods): start(), brake(), refuel(), accelerate().

In OOP, you model real-world things like this as objects in your code.

Instead of having 20 separate variables for car colour, car speed etc., everything lives inside one object.

Why Use OOP?

- **Organises code** — related data and functions stay together
- **Reusable** — write a class once, create 1000 objects from it
- **Easier to maintain** — change one class, all objects using it update
- **Closer to real world** — humans think in objects naturally

02

Objects in JavaScript

What is an Object?

An object is a collection of related data and functions. The data is stored in properties, and the functions are called methods.

Example — Creating a Simple Object

```
// Creating an object using curly braces { }
const student = {
  name: "Raj",
  age: 20,
  course: "Computer Engineering",

  // This is a method (function inside an object)
  introduce: function() {
    console.log("Hi, I am " + this.name);
  }
};

// Accessing properties
console.log(student.name);    // "Raj"
console.log(student.age);    // 20

// Calling a method
student.introduce();          // "Hi, I am Raj"
```

Adding and Changing Properties

```
const phone = {
  brand: "Samsung",
  battery: 80
};

// Change a property
phone.battery = 65;

// Add a new property
phone.colour = "black";

console.log(phone.battery);    // 65
console.log(phone.colour);    // "black"
```

Practice Exercise

Create an object called laptop with these properties:

- brand (string)
- ram (number in GB)
- isOn (boolean, starts as false)

Add a method called powerOn() that sets isOn to true and prints "Laptop is ON".

Then call laptop.powerOn() and check laptop.isOn.

03

Classes — Blueprints for Objects

The Problem With Plain Objects

Imagine you need 50 student objects. Writing each one individually would be very tedious and error prone. There must be a better way.

Analogy — Blueprint and Houses

A class is like a blueprint (naksha) for building a house.

The blueprint defines: how many rooms, doors, windows.

From ONE blueprint you can build MANY houses.

Each house is an object (called an instance) created from that blueprint (class).

Creating a Class

```
// Define the class (the blueprint)
class Student {
  constructor(name, age, course) {
    this.name = name;
    this.age = age;
    this.course = course;
  }

  introduce() {
    console.log("Hi, I am " + this.name + ", age " + this.age);
  }
}

// Create objects (instances) from the class
const student1 = new Student("Raj", 20, "Computer Engg");
const student2 = new Student("Priya", 19, "IT");
const student3 = new Student("Aman", 21, "Mechanical");

student1.introduce(); // "Hi, I am Raj, age 20"
student2.introduce(); // "Hi, I am Priya, age 19"

console.log(student1.name); // "Raj"
console.log(student2.name); // "Priya"
```

Key point: Each object created from a class is called an **instance**. `student1`, `student2`, `student3` are all instances of the `Student` class.



Practice Exercise

Create a class called `Car` with:

- Properties: `brand`, `colour`, `speed` (starts at 0)
- Method `accelerate(amount)` — increases speed by amount
- Method `brake()` — sets speed to 0
- Method `showSpeed()` — prints current speed

Create 2 car objects and test all methods.

04

The Constructor

What is a Constructor?

The constructor is a special method inside a class that runs automatically when you create a new object using the `new` keyword. Its job is to set up (initialise) the object with its starting values.



Analogy — Factory Setup

Think of a constructor as the factory worker who sets up each product when it comes off the assembly line.

When you say `new Car("Toyota", "Red")`, the constructor immediately sets `brand = "Toyota"` and `colour = "Red"` on that specific car object.

Constructor in Detail

```
class BankAccount {
  constructor(ownerName, initialBalance) {
    // "this" refers to the NEW object being created
    this.owner = ownerName;
    this.balance = initialBalance;
    this.transactions = []; // empty array for every new account
  }

  deposit(amount) {
    this.balance = this.balance + amount;
    this.transactions.push("Deposited: " + amount);
    console.log("New balance: " + this.balance);
  }

  showBalance() {
    console.log(this.owner + " has Rs." + this.balance);
  }
}

const acc1 = new BankAccount("Raj", 5000);
const acc2 = new BankAccount("Priya", 12000);

acc1.deposit(2000); // "New balance: 7000"
acc1.showBalance(); // "Raj has Rs.7000"
acc2.showBalance(); // "Priya has Rs.12000"
```

Practice Exercise

Create a class called Movie with constructor taking: title, director, year.

Add a method describe() that prints all three details.

Create 3 movie objects for your favourite movies and call describe() on each.

05

The "this" Keyword

Why Does "this" Confuse Everyone?

"this" is probably the most confusing thing in JavaScript OOP. But the rule is actually very simple once you understand it.

👉 The One Rule for "this"

"this" always refers to the object that is calling the method.

When you write `acc1.deposit()`, inside `deposit()`, `"this" = acc1`.

When you write `acc2.deposit()`, inside `deposit()`, `"this" = acc2`.

"this" is how each object refers to itself.

Seeing "this" in Action

```
class Person {
  constructor(name, age) {
    this.name = name;    // "this" = the new Person being created
    this.age = age;
  }

  greet() {
    // "this" here = whatever object called greet()
    console.log("Hello, I am " + this.name);
  }

  isAdult() {
    return this.age >= 18; // "this.age" = this object's age
  }
}

const p1 = new Person("Raj", 20);
const p2 = new Person("Priya", 16);

p1.greet();           // "Hello, I am Raj"
p2.greet();           // "Hello, I am Priya"

console.log(p1.isAdult()); // true  (20 >= 18)
console.log(p2.isAdult()); // false (16 >= 18)
```

Common "this" Mistake

```
class Timer {
  constructor() {
    this.seconds = 0;
  }

  // ✗ WRONG - regular function loses "this"
  startWrong() {
    setInterval(function() {
      this.seconds++; // ERROR: "this" is undefined here!
    }, 1000);
  }

  // ✓ CORRECT - arrow function keeps "this"
  startCorrect() {
    setInterval(() => {
      this.seconds++; // Works! Arrow function inherits "this"
      console.log(this.seconds);
    }, 1000);
  }
}
```

💡 Quick Rule

Inside a class method, always use arrow functions (`() => {}`) for callbacks.

Arrow functions do NOT create their own "this", they borrow it from the surrounding code.

✎ Practice Exercise

Create a class Counter with:

- Property: count (starts at 0)
- Method increment() — adds 1 to count, prints new count
- Method decrement() — subtracts 1 from count, prints new count
- Method reset() — sets count back to 0

Create two separate Counter objects and prove they have INDEPENDENT counts.

06

Encapsulation

What is Encapsulation?

Encapsulation means bundling the data (properties) and the methods that work on that data together inside a class — AND protecting sensitive data from being changed from outside.

Analogy — A Medicine Capsule

A medicine capsule bundles all the ingredients together inside a shell.

You cannot directly touch or mix the ingredients inside.

You just use it as intended — you swallow it.

Similarly, an object hides its internal details and only exposes what is needed.

The Problem Without Encapsulation

```
// Without encapsulation - anyone can break your object
const account = {
  balance: 5000
};

// Oops - someone set balance to a negative number directly!
account.balance = -99999; // This should NOT be allowed!
console.log(account.balance); // -99999 (very bad)
```

The Solution — Private Properties (using #)

```
class BankAccount {
  #balance;    // # makes this property PRIVATE

  constructor(initialBalance) {
    if (initialBalance < 0) {
      throw new Error("Balance cannot be negative!");
    }
    this.#balance = initialBalance;
  }

  // Public method to safely deposit
  deposit(amount) {
    if (amount <= 0) {
      console.log("Deposit amount must be positive");
      return;
    }
    this.#balance += amount;
    console.log("Deposited. New balance: " + this.#balance);
  }

  // Public method to safely withdraw
  withdraw(amount) {
    if (amount > this.#balance) {
      console.log("Not enough balance!");
      return;
    }
    this.#balance -= amount;
    console.log("Withdrawn. Remaining: " + this.#balance);
  }

  // Public method to READ balance (but not set it)
  getBalance() {
    return this.#balance;
  }
}

const acc = new BankAccount(5000);
acc.deposit(2000);           // "Deposited. New balance: 7000"
acc.withdraw(1000);         // "Withdrawn. Remaining: 6000"

// Try to access private property directly
console.log(acc.#balance);  // ✗ ERROR — cannot access private property
console.log(acc.getBalance()); // ✔ 6000 — use the public method instead
```

Practice Exercise

Create a class PasswordManager with:

- A private property #password

- Method setPassword(newPass) — sets the password (min 8 characters, reject if shorter)
- Method checkPassword(attempt) — returns true if attempt matches, false otherwise
- NO method that directly returns the password

Test it by setting a password and checking with correct and wrong attempts.

07

Abstraction

What is Abstraction?

Abstraction means hiding complex internal details and only showing what is necessary. You expose simple methods, and the complex logic stays hidden inside.

Analogy — Your Smartphone

When you tap "Send" on WhatsApp, you do not need to know about:

- Network protocols, encryption, data packets, server routing...

You just tap Send. The complexity is abstracted away.

You only see the simple interface. The complex work happens behind the scenes.

Abstraction in Code

```
class EmailSender {
  // These are complex methods – users of this class do not need to know
  // how they work
  #connectToServer() {
    console.log("Connecting to SMTP server...");
    // Imagine complex network connection code here
  }

  #formatEmail(to, subject, body) {
    return { to, subject, body, timestamp: new Date() };
    // Imagine complex email formatting code here
  }

  #disconnectFromServer() {
    console.log("Disconnecting...");
  }

  // This is the SIMPLE public method the user calls
  sendEmail(to, subject, body) {
    this.#connectToServer();
    const email = this.#formatEmail(to, subject, body);
    console.log("Email sent to: " + email.to);
    this.#disconnectFromServer();
  }
}

const mailer = new EmailSender();
// User only needs to know about sendEmail()
// All the complex steps are hidden
mailer.sendEmail("raj@example.com", "Hello", "How are you?");
```

Encapsulation vs Abstraction: *Encapsulation is about protecting data (using private). Abstraction is about hiding complexity (showing only what is needed).*



Practice Exercise

Create a class ATM with:

- Private methods: #verifyPin(pin), #checkBalance(amount), #dispenseCash(amount)
- One simple public method: withdraw(pin, amount)
 - it calls the private methods internally
- The user only calls withdraw() — they never call the private methods directly

08

Inheritance

What is Inheritance?

Inheritance allows one class to get all the properties and methods of another class. The class that gives is called the Parent class. The class that receives is called the Child class.



Analogy — Parent and Child

A child inherits things from their parent — eye colour, height, certain traits.

But the child can also have their own unique traits that the parent does not have.

In the same way, a child class gets everything from the parent class, and can also add its own extra properties and methods.

Inheritance Example

```
// PARENT CLASS - general animal
class Animal {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  eat() {
    console.log(this.name + " is eating.");
  }

  sleep() {
    console.log(this.name + " is sleeping.");
  }
}

// CHILD CLASS - Dog extends Animal
class Dog extends Animal {
  constructor(name, age, breed) {
    super(name, age); // Call parent constructor first
    this.breed = breed; // Dog has extra property
  }

  bark() { // Dog has extra method
    console.log(this.name + " says: Woof!");
  }
}

// CHILD CLASS - Cat extends Animal
class Cat extends Animal {
  constructor(name, age, indoor) {
    super(name, age);
    this.indoor = indoor;
  }

  meow() {
    console.log(this.name + " says: Meow!");
  }
}

const dog = new Dog("Bruno", 3, "Labrador");
const cat = new Cat("Whiskers", 2, true);

dog.eat(); // Inherited from Animal - "Bruno is eating."
dog.sleep(); // Inherited from Animal - "Bruno is sleeping."
dog.bark(); // Dog's own method - "Bruno says: Woof!"

cat.eat(); // Inherited from Animal - "Whiskers is eating."
cat.meow(); // Cat's own method - "Whiskers says: Meow!"
```

The super() Keyword

super() calls the parent class's constructor. You **MUST** call **super()** before using "this" in a child class constructor. It passes the required data up to the parent.

Practice Exercise

Create a parent class Vehicle with: brand, speed, and a move() method.

Create two child classes:

- Bicycle — extends Vehicle, add gears property and changeGear(n) method
- Motorcycle — extends Vehicle, add engineCC property and wheelie() method

Both should be able to call move() from Vehicle, plus their own methods.

09

Polymorphism

What is Polymorphism?

Poly = many, Morph = forms. Polymorphism means the same method name behaves differently depending on the object calling it.

Analogy — speak()

Consider a method called speak().

If a Dog calls speak() — it barks.

If a Cat calls speak() — it meows.

If a Human calls speak() — it talks.

Same method name, different behaviour. That is polymorphism.

Polymorphism in Code

```
class Shape {
  area() {
    return 0;    // default — will be overridden
  }

  describe() {
    console.log("This shape has area: " + this.area());
  }
}

class Circle extends Shape {
  constructor(radius) {
    super();
    this.radius = radius;
  }

  area() {    // overrides parent's area()
    return 3.14 * this.radius * this.radius;
  }
}

class Rectangle extends Shape {
  constructor(width, height) {
    super();
    this.width = width;
    this.height = height;
  }

  area() {    // overrides parent's area()
    return this.width * this.height;
  }
}

const shapes = [new Circle(5), new Rectangle(4, 6), new Circle(3)];

// Same method call, different results based on object type
shapes.forEach(shape => shape.describe());
// "This shape has area: 78.5"
// "This shape has area: 24"
// "This shape has area: 28.26"
```

Practice Exercise

Create a parent class Employee with: name, salary, and a getDetails() method.

Create child classes:

- Manager — getDetails() shows name + salary + "Department Head"
- Developer — getDetails() shows name + salary + "Software Developer"

- Intern — `getDetails()` shows name + salary + "Intern (6 months)"
- Put all three in an array and loop through calling `getDetails()` on each.

10

Method Overriding

What is Method Overriding?

When a child class defines a method with the SAME name as a method in the parent class, the child's version replaces (overrides) the parent's version for that child's objects.

Method Overriding Example

```
class Notification {
  send(message) {
    console.log("Sending notification: " + message);
  }
}

class EmailNotification extends Notification {
  send(message) { // Overrides parent's send()
    console.log("EMAIL: " + message + " (from: noreply@app.com)");
  }
}

class SMSNotification extends Notification {
  send(message) { // Overrides parent's send()
    console.log("SMS to +91XXXXXXXX: " + message);
  }
}

class PushNotification extends Notification {
  send(message) { // Overrides parent's send()
    console.log("PUSH ALERT: " + message + " 🔔");
  }
}

const email = new EmailNotification();
const sms = new SMSNotification();
const push = new PushNotification();

email.send("Your order is confirmed!");
// "EMAIL: Your order is confirmed! (from: noreply@app.com)"

sms.send("OTP: 4829");
// "SMS to +91XXXXXXXX: OTP: 4829"

push.send("Flash Sale starts now!");
// "PUSH ALERT: Flash Sale starts now! 🔔"
```

Calling the Parent Method Too — using super

```
class Logger {
  log(message) {
    console.log("[LOG] " + message);
  }
}

class TimestampLogger extends Logger {
  log(message) {
    const time = new Date().toLocaleTimeString();
    super.log(message); // Calls parent's log() first
    console.log("  ^ logged at: " + time);
  }
}

const tLogger = new TimestampLogger();
tLogger.log("User logged in");
// "[LOG] User logged in"
// "  ^ logged at: 3:45:21 PM"
```

11

Method Overloading (JavaScript Style)

Does JavaScript Support Overloading?

In languages like Java or C++, you can have multiple methods with the same name but different parameters. JavaScript does NOT support this directly. If you define the same method name twice, the second one simply replaces the first.

However, JavaScript gives you a way to achieve the same result using default parameters and checking argument types inside one method.

JavaScript Overloading Pattern

```
class Calculator {  
  // One method that handles different cases  
  add(a, b = 0, c = 0) {  
    // b and c have default value of 0  
    return a + b + c;  
  }  
}  
  
const calc = new Calculator();  
console.log(calc.add(5));           // 5 (only a)  
console.log(calc.add(5, 3));        // 8 (a + b)  
console.log(calc.add(5, 3, 2));     // 10 (a + b + c)
```

```
// More advanced - handle completely different input types  
class Greeter {  
  greet(input) {  
    if (typeof input === "string") {  
      console.log("Hello, " + input + "!");  
    } else if (Array.isArray(input)) {  
      input.forEach(name => console.log("Hello, " + name + "!"));  
    } else if (typeof input === "object") {  
      console.log("Hello, " + input.firstName + " " + input.lastName +  
        "!");  
    }  
  }  
}  
  
const g = new Greeter();  
g.greet("Raj");                     // "Hello, Raj!"  
g.greet(["Raj", "Priya", "Aman"]);  // Hello to all three  
g.greet({ firstName: "Raj", lastName: "Sharma" }); // "Hello, Raj  
Sharma!"
```

12

Prototypes & Prototype Chain

What is a Prototype?

This is the "under the hood" part of JavaScript OOP. Every object in JavaScript has a hidden link to another object called its prototype. When you call a method on an object and JS cannot find it on that object, it automatically looks up to its prototype.

🔗 Analogy — The Family Family Tree

If you ask a child "can you speak English?" and the child cannot, JavaScript asks the parent. If the parent cannot, it asks the grandparent. This chain of looking upward is the prototype chain. It keeps going until it reaches null (no more ancestors).

Prototype in Action

```
// When you use class syntax, JS sets up prototypes automatically
class Dog {
  constructor(name) {
    this.name = name;
  }

  bark() {
    console.log(this.name + " barks!");
  }
}

const dog = new Dog("Bruno");

// bark() is NOT stored on the dog object itself
// It is stored on Dog.prototype
// When you call dog.bark(), JS:
// 1. Looks for bark on dog object — not found
// 2. Looks for bark on Dog.prototype — FOUND!
// 3. Calls it

console.log(dog.hasOwnProperty("name")); // true — stored on dog
console.log(dog.hasOwnProperty("bark")); // false — stored on prototype
```

The Old Way — Prototypes Without Class Syntax

```
// BEFORE ES6 classes existed, this is how OOP was done
// You will see this in older code

function Person(name, age) {
  this.name = name;
  this.age = age;
}

// Methods added to the prototype (shared by all instances)
Person.prototype.greet = function() {
  console.log("Hi, I am " + this.name);
};

const p1 = new Person("Raj", 20);
const p2 = new Person("Priya", 19);

p1.greet(); // "Hi, I am Raj"
p2.greet(); // "Hi, I am Priya"

// Both p1 and p2 share the SAME greet function from Person.prototype
// This saves memory — the function is not duplicated for each object
```

ES6 Classes vs Prototype-Based OOP

	ES6 Class Syntax	Old Prototype Way
Readability	Very clean and readable	Harder to read for beginners
Is it really OOP	Looks like Java/Python OOP	Same thing, just different syntax
Under the hood	Uses prototypes internally	Uses prototypes directly
Modern code	Used in all modern projects	Found in older codebases
Use it?	✔ Yes — always use class syntax	🔍 Understand it to read old code

13

Static Methods

What are Static Methods?

A static method belongs to the CLASS itself, not to any individual object. You call it on the class name, not on an instance.



Analogy — Factory Rules vs Product Features

The Car factory has a rule: "We only build red and blue cars."

That rule belongs to the FACTORY, not to any individual car.

Static methods belong to the class (factory), not to objects (cars).

Static Methods Example

```
class MathHelper {
  // Static method - belongs to the class, not instances
  static square(n) {
    return n * n;
  }

  static cube(n) {
    return n * n * n;
  }

  static isEven(n) {
    return n % 2 === 0;
  }
}

// Call on the CLASS NAME - not on an object
console.log(MathHelper.square(4));    // 16
console.log(MathHelper.cube(3));      // 27
console.log(MathHelper.isEven(7));    // false

// You do NOT do this:
const m = new MathHelper();
m.square(4);    // ❌ ERROR - static methods cannot be called on instances
```

```
// Real-world use: counter of how many objects were created
class User {
  static count = 0;    // static property

  constructor(name) {
    this.name = name;
    User.count++;      // increment on class, not on this
  }

  static getCount() {
    return User.count;
  }
}

const u1 = new User("Raj");
const u2 = new User("Priya");
const u3 = new User("Aman");

console.log(User.getCount());    // 3
```

14

Getters and Setters

What are Getters and Setters?

Getters and Setters let you access and change properties of an object using simple property syntax (no parentheses), while running code behind the scenes. They are like smart properties.

💰 Analogy — ATM Display

You look at the ATM screen and it shows "Balance: Rs. 5,000" — that is a getter.

You type in a new PIN — the ATM validates it first before saving — that is a setter.

The getter/setter can run code (validation, formatting) while looking like a simple property.

Getters and Setters Example

```
class Temperature {
  #celsius;

  constructor(celsius) {
    this.#celsius = celsius;
  }

  // GETTER - accessed like a property, no ()
  get fahrenheit() {
    return (this.#celsius * 9/5) + 32;
  }

  get celsius() {
    return this.#celsius;
  }

  // SETTER - validates before setting
  set celsius(value) {
    if (value < -273.15) {
      throw new Error("Temperature below absolute zero is impossible!");
    }
    this.#celsius = value;
  }
}

const temp = new Temperature(100);

console.log(temp.celsius);      // 100 - no () needed
console.log(temp.fahrenheit);   // 212 - calculated automatically

temp.celsius = 37;              // uses the setter - no () needed
console.log(temp.fahrenheit);   // 98.6

temp.celsius = -300;            // ✗ throws error - setter validates
```



Practice Exercise

Create a class Circle with a private property #radius.

Add:

- A setter for radius that rejects negative values
- A getter for radius that returns the value
- A getter for area that calculates and returns $3.14 * r * r$
- A getter for circumference that calculates $2 * 3.14 * r$

Access area and circumference without calling them as functions.

15

Private Properties (Deep Dive)

Everything About Private Properties

You already saw private properties in the Encapsulation section. Here is the complete picture.

Private Properties and Methods with

```

class SecureVault {
  // Private property - cannot be accessed outside the class
  #pin;
  #balance;
  #transactionHistory = [];

  constructor(pin, initialBalance) {
    this.#pin = pin;
    this.#balance = initialBalance;
  }

  // Private method - internal helper
  #logTransaction(type, amount) {
    this.#transactionHistory.push({
      type,
      amount,
      time: new Date().toLocaleTimeString()
    });
  }

  // Public method - exposed to outside
  deposit(pin, amount) {
    if (pin !== this.#pin) {
      console.log("Wrong PIN!");
      return;
    }
    this.#balance += amount;
    this.#logTransaction("deposit", amount); // calls private method
    console.log("Deposited. Balance: " + this.#balance);
  }

  getHistory(pin) {
    if (pin !== this.#pin) {
      console.log("Wrong PIN!");
      return;
    }
    return this.#transactionHistory;
  }
}

const vault = new SecureVault("1234", 10000);
vault.deposit("1234", 5000); // works
vault.deposit("0000", 5000); // "Wrong PIN!"

console.log(vault.#pin); // ✗ ERROR - private!
  
```

16 Real-World Project — Library Management System

Project Overview

This project combines everything you have learned. We will build a Library Management System using OOP. It has books, members, and a library that manages them.

 What this project covers:

- Classes and constructors (Book, Member, Library)
- Private properties (book availability, member ID)
- Getters and setters (book info)
- Inheritance (PremiumMember extends Member)
- Static methods (Library.getTotalBooks)
- Encapsulation (borrow/return logic)
- Method overriding (PremiumMember can borrow more books)

```
// — Book Class —
class Book {
  #isAvailable = true;

  constructor(title, author, isbn) {
    this.title = title;
    this.author = author;
    this.isbn = isbn;
  }

  get available() {
    return this.#isAvailable;
  }

  get info() {
    const status = this.#isAvailable ? "Available" : "Borrowed";
    return this.title + " by " + this.author + " [" + status + "];"
  }

  checkOut() {
    if (!this.#isAvailable) {
      console.log(this.title + " is already borrowed.");
      return false;
    }
    this.#isAvailable = false;
    return true;
  }
}
```

```

    returnBook() {
      this.#isAvailable = true;
      console.log(this.title + " has been returned.");
    }
  }

  // — Member Class —
class Member {
  static #nextId = 1;
  #borrowedBooks = [];
  #maxBooks = 3; // regular members can borrow 3

  constructor(name) {
    this.name = name;
    this.id = "M" + Member.#nextId++;
  }

  get borrowedCount() {
    return this.#borrowedBooks.length;
  }

  borrow(book) {
    if (this.#borrowedBooks.length >= this.#maxBooks) {
      console.log(this.name + " has reached borrow limit!");
      return;
    }
    if (book.checkOut()) {
      this.#borrowedBooks.push(book);
      console.log(this.name + " borrowed: " + book.title);
    }
  }

  returnBook(book) {
    const index = this.#borrowedBooks.indexOf(book);
    if (index === -1) {
      console.log(this.name + " did not borrow " + book.title);
      return;
    }
    this.#borrowedBooks.splice(index, 1);
    book.returnBook();
  }

  showBorrowed() {
    console.log(this.name + " has borrowed " + this.borrowedCount + " books.");
    this.#borrowedBooks.forEach(b => console.log("  - " + b.title));
  }
}

// — PremiumMember extends Member —
class PremiumMember extends Member {
  constructor(name) {
    super(name);
  }
}

```

```

    this.memberType = "Premium";
    // Premium members can borrow 6 books - but #maxBooks is private!
    // We handle this via method overriding
  }

  // Override borrow - higher limit for premium
  borrow(book) {
    if (this.borrowedCount >= 6) {
      console.log(this.name + " (Premium) has reached limit of 6 books!");
      return;
    }
    if (book.checkOut()) {
      // Use parent logic via super
      super.borrow(book);
    }
  }
}

// — Library Class —————
class Library {
  #books = [];
  #members = [];
  static #totalLibraries = 0;

  constructor(name) {
    this.name = name;
    Library.#totalLibraries++;
  }

  addBook(book) {
    this.#books.push(book);
    console.log("Added: " + book.title + " to " + this.name);
  }

  registerMember(member) {
    this.#members.push(member);
    console.log("Registered: " + member.name + " (ID: " + member.id + ")");
  }

  findBook(title) {
    return this.#books.find(b => b.title === title);
  }

  showAllBooks() {
    console.log("\n=== Books in " + this.name + " ===");
    this.#books.forEach(b => console.log(" " + b.info));
  }

  static getTotalLibraries() {
    return Library.#totalLibraries;
  }
}

```

```
// — TEST IT —  
const library = new Library("City Library");  
  
const book1 = new Book("Clean Code", "Robert Martin", "ISBN-001");  
const book2 = new Book("JavaScript: Good Parts", "Douglas Crockford",  
"ISBN-002");  
const book3 = new Book("You Don't Know JS", "Kyle Simpson", "ISBN-003");  
  
library.addBook(book1);  
library.addBook(book2);  
library.addBook(book3);  
  
const raj = new Member("Raj");  
const priya = new PremiumMember("Priya");  
  
library.registerMember(raj);  
library.registerMember(priya);  
  
raj.borrow(book1);  
raj.borrow(book2);  
raj.showBorrowed();  
  
library.showAllBooks();  
  
raj.returnBook(book1);  
  
priya.borrow(book1); // now available again  
priya.showBorrowed();  
  
console.log("Total libraries: " + Library.getTotalLibraries());
```

Challenge — Extend This Project

Try adding these features on your own:

1. A fine system — if a book is not returned in 7 days, add Rs.5/day fine
2. A search method — `Library.search(keyword)` that searches by title or author
3. A waitlist — if a book is borrowed, members can join a waiting list
4. Book categories — Fiction, Non-Fiction, Technical — and filter by category

OOP Cheat Sheet — Quick Reference

Concept	One Line Definition	Key Syntax
Class	Blueprint for creating objects	<code>class Car { }</code>
Constructor	Runs automatically when object is created	<code>constructor(name) { this.name = name }</code>
this	Refers to the current object	<code>this.property</code>
new	Creates an instance from a class	<code>const c = new Car("Toyota")</code>
Inheritance	Child class gets parent's properties/methods	<code>class Dog extends Animal { }</code>
super()	Calls the parent class constructor	<code>super(name, age)</code>
Encapsulation	Bundling + protecting data	<code>class with #private and public methods</code>
Abstraction	Hiding complexity, showing simple interface	<code>Private methods + simple public method</code>
Polymorphism	Same method, different behaviour per class	<code>Override method in child class</code>
Private (#)	Cannot be accessed outside the class	<code>#balance; this.#balance</code>
Static	Belongs to class, not to instances	<code>static count = 0; ClassName.count</code>
get / set	Smart properties with code behind them	<code>get area() { return ... }</code>
Prototype	Hidden link all objects have to their creator	<code>Object.__proto__ or ClassName.prototype</code>

You Made It!

If you have read every section and done every exercise, you now fully understand OOP in JavaScript.

The next step: go back to Shraddha's playlist and continue from the classes section. Everything will make complete sense now.

Build something: try the Library project first, then think of your own OOP project idea.

